
**LAPORAN PRAKTIKUM
INTERNET OF THINGS (IOT)**

TUGAS 1



Oleh :

Nama : Rena Anggraini

NPM : 25782022

Kelas : TRI (3A)

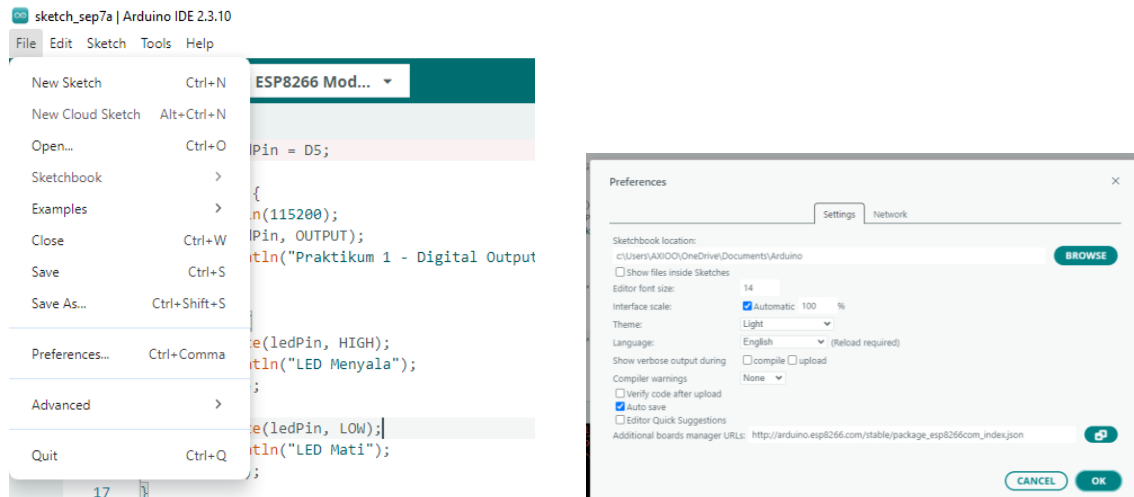
**Dosen Pengampu : Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.
Andi Saptono, A.Md**

**JURUSAN TEKNOLOGI INFORMASI
TEKNOLOGI REKAYASA INTERNET
POLITEKNIK NEGERI LAMPUNG
TAHUN 2026**

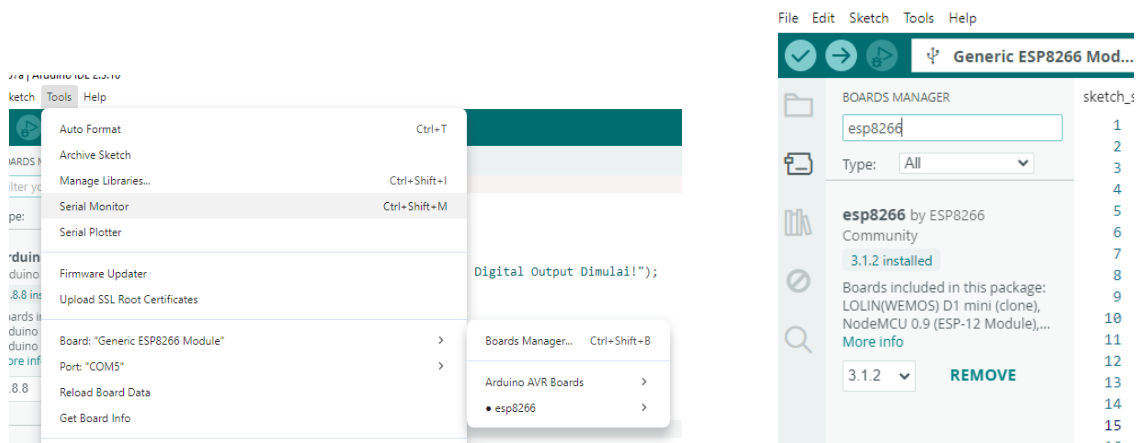
SCREENSHOT

Simulasi Rangkaian Praktikum

Instalasi Apk Arduino EDA

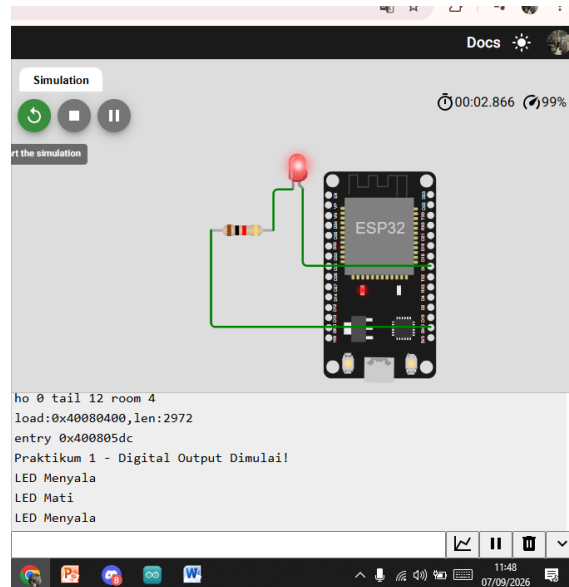


- Buka menu **File** lalu pilih **Preferences** untuk membuka jendela pengaturan utama Arduino IDE.
- Masukkan tautan modul pihak ketiga pada kolom **Additional boards manager URLs**, lalu klik **OK** untuk menyimpan.



- Buka **Tools > Board > Boards Manager**, lalu ketik "esp8266" pada kolom pencarian untuk menemukan paket perangkat keras yang sesuai.
- Pilih dan klik **Install** pada paket **esp8266 by ESP8266 Community** agar Arduino IDE mengenali arsitektur modul tersebut.

Praktikum 1 : Digital Output (Menyalakan LED)



Digital output, untuk menyalakan dan mematikan LED menggunakan mikrokontroler.

- Board Mikrokontroler (ESP), Berfungsi sebagai otak sistem yang memproses dan mengeksekusi kode program untuk mengontrol aliran listrik.
- Lampu LED, Berfungsi sebagai indikator visual (*output*) yang menyala saat dialiri listrik dan mati saat terputus.
- Resistor, Berfungsi sebagai pembatas arus listrik agar LED tidak kelebihan daya dan terbakar.
- Kabel Pin Mikrokontroler ke LED: Berfungsi sebagai jalur pengiriman sinyal digital (tegangan 3.3V) dari program untuk menghidupkan LED.
- Kabel Resistor ke Pin GND, Berfungsi sebagai jalur penutup rangkaian (*ground*) untuk mengalirkan kembali sisa arus listrik ke mikrokontroler.

```
WOKWI SAVE SHARE ESP8266 ESP8266WiFi by sunu
sketch.ino diagram.json Library Manager
1 const int ledPin = 5;
2
3 void setup() {
4   Serial.begin(115200);
5   pinMode(ledPin, OUTPUT);
6   Serial.println("Praktikum 1 - Digital Output Dimulai!");
7 }
8
9 void loop() {
10  digitalWrite(ledPin, HIGH);
11  Serial.println("LED Menyala");
12  delay(1000);
13
14  digitalWrite(ledPin, LOW);
15  Serial.println("LED Mati");
16  delay(1000);
17 }
```

Code file sketch.ino

```
1 {
2   "version": 1,
3   "author": "Anonymous maker",
4   "editor": "wokwi",
5   "parts": [
6     { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": -14.5, "left": -3
7     {
8       "type": "wokwi-led",
9       "id": "led1",
10      "top": 0.4,
11      "left": 217.8,
12      "rotate": 90,
13      "attrs": { "color": "red" }
14    },
15    {
16      "type": "wokwi-resistor",
17      "id": "r1",
18      "top": 23.15,
19      "left": 124.8,
20      "attrs": { "value": "1000" }
21    }
22  ],
23  "connections": [
24    [ "esp:TX0", "SerialMonitor:RX", "", [] ],
25    [ "esp:RX0", "SerialMonitor:TX", "", [] ],
26    [ "led1:A", "esp:IO5", "green", [ "ho" ] ],
27    [ "led1:C", "r1:2", "green", [ "ho" ] ],
28    [ "r1:1", "esp:GND.1", "green", [ "v0" ] ]
29  ],
30  "dependencies": {}
}
```

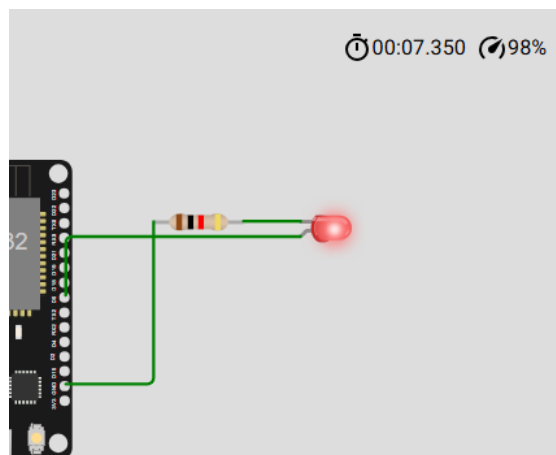
Code diagram.json

Kode program *sketch.ino* berfungsi untuk mengontrol nyala LED secara berkedip setiap satu detik melalui pin digital 5 sekaligus mencetak status teks pada monitor serial, sementara kode *diagram.json* berfungsi untuk mendefinisikan seluruh komponen perangkat keras yang digunakan dalam simulasi—seperti mikrokontroler ESP32, LED merah, dan resistor—beserta jalur sambungan kabel antar komponen tersebut.

Kode program dan perangkat keras pada simulasi telah berhasil saling terhubung dan bekerja dengan sempurna.

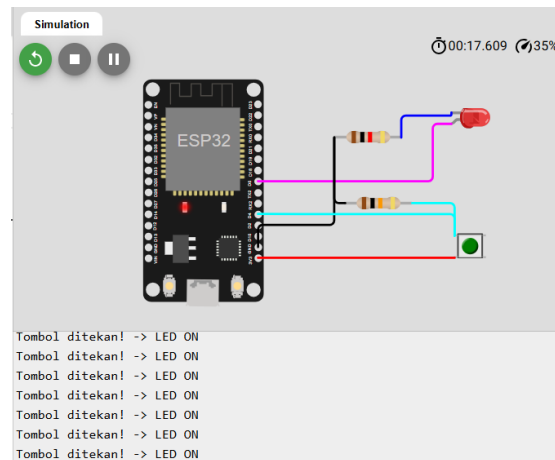


```
12:21:46 00:48.304 98%
rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0xee
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:2
load:0x3fff0030,len:1156
load:0x40078000,len:11456
ho 0 tail 12 room 4
load:0x40080400,len:2972
entry 0x400805dc
Praktikum 1 - Digital Output Dimulai!
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
```



Kode program dan perangkat pada simulasi Wokwi bisa saling terhubung karena adanya kecocokan antara nomor pin yang didefinisikan dalam skrip (seperti `ledPin = 5`) dengan jalur kabel hijau yang menghubungkan pin mikrokontroler ke komponen LED, di mana fungsi `pinMode` dan `digitalWrite` di dalam kode bertindak sebagai pengirim sinyal listrik untuk mengontrol nyala lampu sekaligus menampilkan status teks secara sinkron pada konsol saat tombol simulasi dijalankan.

Praktikum 2: Digital Input (Membaca Push Button)



Simulasi ini merupakan uji coba rangkaian tombol tekan (*push button*) dengan mode *momentary* (tidak mengunci) menggunakan mikrokontroler ESP32, di mana hasil output pada konsol mencetak teks "Tombol ditekan! -> LED ON" secara berulang-ulang dan terus-menerus tanpa berhenti. Perangkat akan berfungsi menyalakan LED hanya selama tombol fisik ditekan dan ditahan oleh pengguna, dan akan langsung mati kembali begitu tombol tersebut dilepaskan karena logika kodenya menggunakan perintah `digitalRead` langsung di dalam fungsi `loop()`.

```
sketch.ino • diagram.json • Library Manager
1  const int buttonPin = 4;
2  const int ledPin = 5;
3
4  int buttonState = 0;
5
6  void setup() {
7    Serial.begin(115200);
8    pinMode(buttonPin, INPUT);
9    pinMode(ledPin, OUTPUT);
10 }
11
12 void loop() {
13   buttonState = digitalRead(buttonPin);
14
15   if (buttonState == HIGH) {
16     digitalWrite(ledPin, HIGH);
17     Serial.println("Tombol ditekan! -> LED ON");
18   } else {
19     digitalWrite(ledPin, LOW);
20   }
21 }
```

Code sketc.ino

```

1  {
2    "version": 1,
3    "author": "Anonymous maker",
4    "editor": "wokwi",
5    "parts": [
6      { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 4.7, "left": -139
7      {
8        "type": "wokwi-led",
9        "id": "led1",
10       "top": 15.6,
11       "left": 128.6,
12       "rotate": 90,
13       "attrs": { "color": "red" }
14     },
15     {
16       "type": "wokwi-resistor",
17       "id": "r1",
18       "top": 51.95,
19       "left": 28.8,
20       "attrs": { "value": "1000" }
21     },
22     {
23       "type": "wokwi-resistor",
24       "id": "r2",
25       "top": 109.55,
26       "left": 38.4,
27       "attrs": { "value": "10000" }
28     },
29     {
30       "type": "wokwi-pushbutton-6mm",
31       "id": "btn1",
32       "top": 141.8,
33       "left": 134.4,
34       "attrs": { "color": "green", "xray": "1" }
35     }
36   ],
37   "connections": [
38     [ "esp:TX0", "$serialMonitor:RX", "", [ ] ],
39     [ "esp:RX0", "$serialMonitor:TX", "", [ ] ],
40     [ "esp:D5", "led1:A", "magenta", [ "h153.3", "v-50.4" ] ],
41     [ "led1:C", "r1:2", "blue", [ "v2.8", "h-44" ] ],
42     [ "r1:1", "esp:GND.1", "black", [ "v76.8", "h-66.9" ] ],
43     [ "esp:3V3", "btn1:2.1", "red", [ "v0", "h172.5" ] ],
44     [ "r2:2", "btn1:1.1", "cyan", [ "v0", "h37.2" ] ],
45     [ "btn1:1.1", "esp:D4", "cyan", [ "v-19.2", "h-134.4", "v-0.1" ] ],
46     [ "r1:1", "r2:1", "black", [ "v0" ] ]
47   ],
48   "dependencies": {}
49 }

```

code diagram.json

```

24   "id": "r2",
25   "top": 109.55,
26   "left": 38.4,
27   "attrs": { "value": "10000" }
28 },
29 {
30   "type": "wokwi-pushbutton-6mm",
31   "id": "btn1",
32   "top": 141.8,
33   "left": 134.4,
34   "attrs": { "color": "green", "xray": "1" }
35 }
36 ],
37 "connections": [
38   [ "esp:TX0", "$serialMonitor:RX", "", [ ] ],
39   [ "esp:RX0", "$serialMonitor:TX", "", [ ] ],
40   [ "esp:D5", "led1:A", "magenta", [ "h153.3", "v-50.4" ] ],
41   [ "led1:C", "r1:2", "blue", [ "v2.8", "h-44" ] ],
42   [ "r1:1", "esp:GND.1", "black", [ "v76.8", "h-66.9" ] ],
43   [ "esp:3V3", "btn1:2.1", "red", [ "v0", "h172.5" ] ],
44   [ "r2:2", "btn1:1.1", "cyan", [ "v0", "h37.2" ] ],
45   [ "btn1:1.1", "esp:D4", "cyan", [ "v-19.2", "h-134.4", "v-0.1" ] ],
46   [ "r1:1", "r2:1", "black", [ "v0" ] ]
47 ],
48 "dependencies": {}
49 }

```

Lanjutan code diagram.json

Kode program *sketch.ino* berfungsi untuk membaca status tombol tekan (push button) pada pin 4 untuk menyalakan atau mematikan LED di pin 5 secara langsung, sedangkan kode *diagram.json* berfungsi untuk memetakan seluruh perangkat keras rangkaian yang mencakup mikrokontroler ESP32, satu LED, dua resistor (1k dan 10k), serta sebuah tombol tekan beserta jalur kabel penghubungnya.

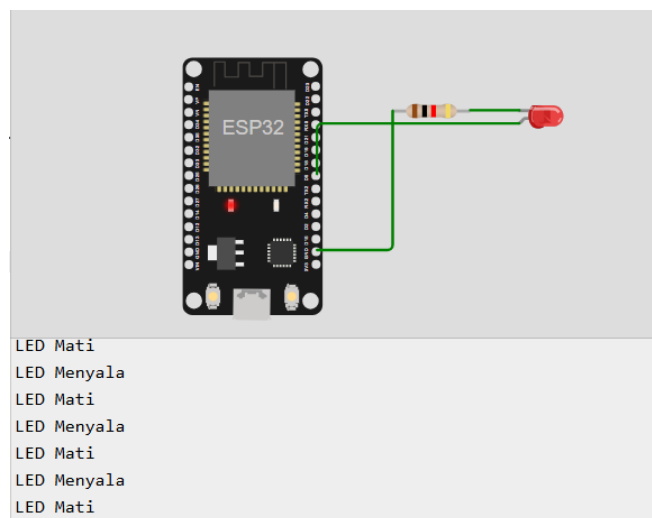
1.6 Latihan

Kerjakan latihan berikut untuk menguji pemahaman Anda. Jelaskan hasilnya di dalam laporan!

- **Latihan 1 (Pemahaman Logika Pin):** Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / *pull-down*), namun menyambungkan tombol tersebut ke pin **D8 (GPIO 15)**, lalu menekan tombol tersebut *tepat* pada detik di mana NodeMCU baru diberi daya?

Jawab : Jika menghubungkan tombol dengan rangkaian pull-down ke pin D8 (GPIO 15) pada NodeMCU dan menekannya tepat saat perangkat baru dinyalakan, mikrokontroler akan gagal melakukan *booting* secara normal karena pin D8 merupakan pin *bootstrapping* khusus yang wajib berada dalam kondisi *LOW* (terhubung ke Ground) saat *power-up* agar sistem dapat memuat program dari memori internal, sehingga tegangan *HIGH* dari tombol yang ditekan saat menyala akan membuat chip mengira ingin masuk ke mode *UART Download* dan menghentikan jalannya program utama seperti pada penugasan.

- **Latihan 2 (Implementasi Kode):** Modifikasi kode pada Praktikum 1 agar LED berkedip secara *tidak simetris*, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi `delay()` yang harus Anda ubah.



Jawab: Bagian yang diubah, nilai waktu di dalam dua baris fungsi `delay()` pada file `sketch.ino`, yaitu mengubah `delay(1000)` menjadi `delay(200)` saat LED menyala dan mengubah `delay(1000)` menjadi `delay(800)` saat LED mati.

Hasil Outpunya, Konsol Serial Monitor mencetak teks "LED Menyala" dan "LED Mati" secara bergantian dengan interval waktu jeda yang tidak sama.

- **Latihan 3 (Analisis Sirkuit):** Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)? Apakah program akan mengalami *error* kompilasi? Mengapa LED tidak menyala?

Jawab: Program tidak akan mengalami kesalahan kompilasi karena compiler hanya memeriksa penulisan kode saja. Sementara itu, LED tidak menyala karena pemasangannya terbalik, sehingga arus listrik terhalang dan tidak dapat mengalir melalui komponen tersebut.

- **Latihan 4 (Pengembangan Algoritma):** Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED **menyala**. Saat tombol DITEKAN, LED justru **mati**."

Jawab: Ubah baris kondisi di dalam fungsi `loop()` pada file `sketch.ino` dari `if (buttonState == HIGH)` menjadi `if (buttonState == LOW)`. Logika tersebut membalik perilaku perangkat sehingga LED menyala saat tombol tidak ditekan dan padam ketika tombol ditekan.

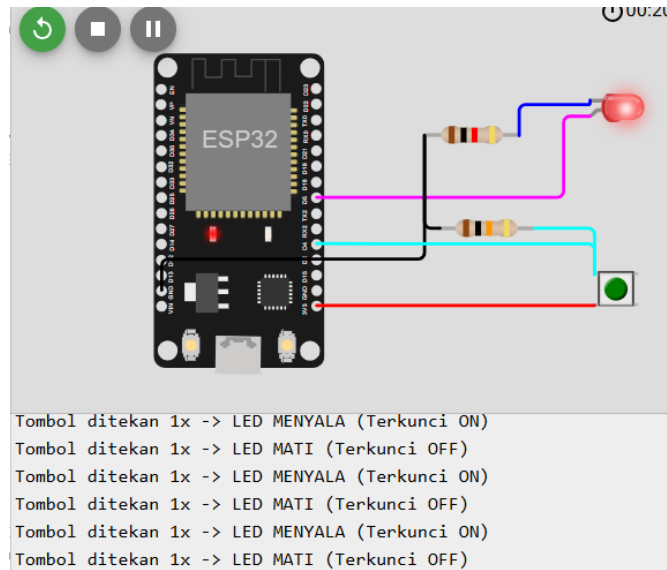
1.7 Tugas

- **Intruksi**

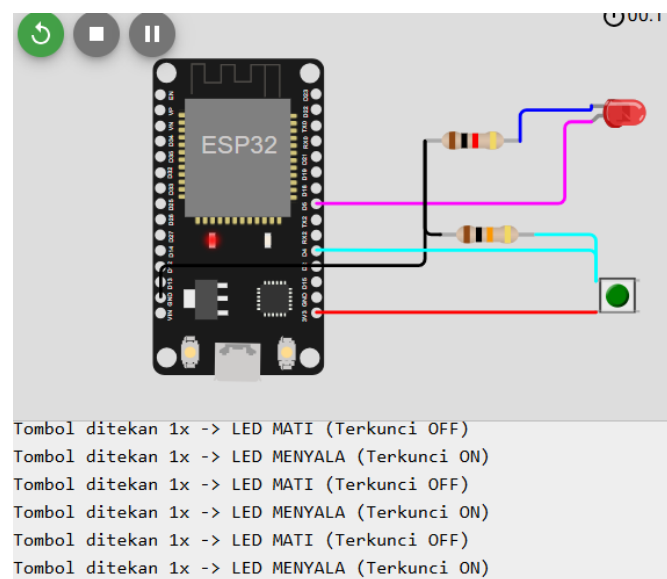
- i. Gunakan konfigurasi *hardware* yang persis sama dengan Praktikum 2 (Rangkaian *Pull-Down*).
- ii. Modifikasi kode program sehingga tombol berfungsi layaknya sakelar lampu ruangan:
 - Saat sistem pertama di nyalakan, LED dalam keadaan mati.
 - Jika tombol di tekan 1 kali lalu di lepas, LED menyala dan tetap menyala.
 - Jika tombol di tekan 1 kali lagi lalu di lepas, LED mati dan tetap mati.
- iii. *Petunjuk*: Gunakan variabel penyimpanan status (misalnya tipe boolean atau int) untuk melacak apakah lampu saat ini sedang ON atau OFF. Anda juga membutuhkan instruksi `delay()` singkat (sekitar 200 milidetik) tepat setelah mendeteksi tombol ditekan, untuk mengatasi fenomena pantulan mekanis (*debouncing* pada tombol fisik).

- **Kriteria Keberhasilan:**

- i. Perubahan status (ON ke OFF, atau sebaliknya) hanya terjadi dengan *satu kali tekanan singkat* (bukan ditahan).
- ii. Tidak ada fenomena *flickering* (lampu berkedip acak atau gagal mati) saat tombol ditekan.
- iii. Program memiliki baris komentar yang menjelaskan bagaimana logika pelacakan *state* dan *debounce* bekerja.



Jika tombol pushbutton di tekan 1x untuk di nyalakan maka lampu LED akan menyala.



Dan jika tombol pushbutton ditekan 1x lagi untuk di matikan maka LED akan mati .

Pembahasan logika program pada laporan :

- Program menggunakan logika *toggle* (sakelar berganti status) di mana penekanan tombol satu kali mengubah status LED dari mati menjadi menyala (terkunci ON).
- Penekanan tombol berikutnya membalikkan status LED kembali menjadi mati (terkunci OFF) berdasarkan pembacaan sinyal input dari rangkaian *pushbutton*.